

ALVA'S EDUCATION FOUNDATION

(Affiliated to Visvesvaraya Technological University, Belagavi)



**ALVA'S INSTITUTE OF ENGINEERING AND TECHNOLOGY
DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING**

A FINAL YEAR PROJECT REPORT ON

PACKSCI AI

**“AI-Based Intelligent Food Packaging Material Recommendation and
RAG-Powered Decision Support System for Food Commodities”**

Submitted by

MOKSHITH H C

USN: 4AL23AI031

LIKITH T M

USN: 4AL23CS070

Bachelor of Engineering

Department of Artificial Intelligence and Machine Learning

Project Guide:

Anjit Raja R

Academic Year: 2026–2027

CERTIFICATE

This is to certify that the project report entitled “PackSci AI: AI-Based Intelligent Food Packaging Material Recommendation and RAG-Powered Decision Support System for Food Commodities” is a bonafide work carried out by **Mokshith H C** (USN: 4AL23AI031) and **Likith T M** (USN: 4AL23CS070) in partial fulfilment of the requirements for the award of the degree of **Bachelor of Engineering in Artificial Intelligence and Machine Learning** of Visvesvaraya Technological University, Belagavi, during the academic year 2026–2027, at Alva's Institute of Engineering and Technology, under the guidance of the undersigned.

The report has been approved as it satisfies the academic requirements in respect of the project work prescribed for the said degree. The work embodied in this report has not been submitted to any other institution for the award of any degree or diploma.

DECLARATION

We, **Mokshith H C** (USN: 4AL23AI031) and **Likith T M** (USN: 4AL23CS070), students of the Department of Artificial Intelligence and Machine Learning, Alva's Institute of Engineering and Technology, hereby declare that the project report entitled “PackSci AI: AI-Based Intelligent Food Packaging Material Recommendation and RAG-Powered Decision Support System for Food Commodities” is an original work carried out by us under the guidance of [Project Guide Name]. This work has not been submitted, in part or in full, to any other university or institution for the award of any degree, diploma or certificate.

We further declare that all sources of information, tools, libraries and references used during this work have been acknowledged, and that where results are described as prototype outputs or reference values, they are not presented as experimental laboratory measurements.

Place: Mijar

Date:

Mokshith H C
Likith T M

ACKNOWLEDGEMENT

We take this opportunity to thank everyone who supported us during this project. We are grateful to Alva's Education Foundation and the management of Alva's Institute of Engineering and Technology for providing an environment that encourages learning and practical problem solving.

We sincerely thank the Principal and the Head of the Department of Artificial Intelligence and Machine Learning for their encouragement and for the facilities made available to us. We express our deep gratitude to our project guide, [Project Guide Name], for valuable guidance, patient reviews and timely feedback from problem definition to completion of this report.

We also thank the faculty and staff of the department for their cooperation, and the open-source communities behind the tools used in this work. Finally, we thank our parents, family and friends for their constant support and motivation.

Mokshith H C
Likith T M

ABSTRACT

Selecting a packaging material for a food product requires balancing barrier properties, storage temperature, shelf-life needs, mechanical demands, cost and sustainability. Manual selection is slow and inconsistent, and general-purpose language models may state packaging properties that no source supports. This report presents PackSci AI, a prototype decision-support platform that addresses both problems.

PackSci AI derives barrier requirements, namely the maximum allowable oxygen and water-vapour transmission rates, from food properties and storage conditions using documented engineering relations, including Fick's law, permeability, respiration kinetics with a configurable Q_{10} assumption, and a lipid-oxidation approximation. A deterministic Multi-Criteria Decision Making engine applies hard constraints and ranks materials on six weighted criteria: moisture barrier, oxygen barrier, thermal compatibility, mechanical performance, sustainability and cost. A separate Retrieval-Augmented Generation layer embeds queries with BAAI/bge-small-en-v1.5, retrieves evidence from Qdrant Cloud, cross-references PostgreSQL records and asks a Groq-hosted language model to produce an explanation that cites its sources and states uncertainty.

The system is implemented with React, TypeScript, Vite and Tailwind CSS on the frontend, and FastAPI, SQLAlchemy and Supabase PostgreSQL on the backend, secured with bcrypt, JWT and role-based access control. The current prototype dataset contains approximately 53 or more food commodity records and 20 or more packaging material records. Four case studies (strawberries, potato chips, fresh chicken and roasted coffee) illustrate the approach. The report does not claim laboratory validation, predictive accuracy or measured shelf-life improvement; it separates literature data, prototype behaviour and future experimental validation, and proposes a validation plan based on ASTM D3985 and ASTM F1249 measurements.

Keywords: *food packaging, packaging material recommendation, multi-criteria decision making, retrieval-augmented generation, oxygen transmission rate, water vapour transmission rate, modified atmosphere packaging, Qdrant, FastAPI, decision support.*

TABLE OF CONTENTS

Certificate	2
Declaration	3
Acknowledgement	4
Abstract	5
List of Tables	8
List of Abbreviations.....	9
Chapter 1: Introduction	10
1.1 Background	10
1.2 Problem Statement.....	10
1.3 Objectives.....	11
1.4 Scope of the Project	11
1.5 Motivation.....	12
1.6 Need for Intelligent Packaging Selection.....	12
1.7 Project Contributions	12
1.8 Report Organization.....	12
Chapter 2: Literature Review	14
2.1 Food Packaging and Shelf Life	14
2.2 Oxygen and Moisture Barrier Requirements.....	14
2.3 Polymer Packaging Materials.....	14
2.4 Modified Atmosphere Packaging	15
2.5 Food Respiration.....	15
2.6 Food Oxidation.....	16
2.7 AI in Food Packaging	16
2.8 Retrieval-Augmented Generation.....	16
2.9 Research Gap.....	16
Chapter 3: System Requirements and Architecture	18
3.1 Functional Requirements	18
3.2 Non-Functional Requirements.....	18
3.3 Technology Stack	19
3.4 Overall System Architecture	19
3.5 Frontend Architecture	20
3.6 Backend Architecture.....	21
3.7 Database Architecture.....	21
3.8 Vector Database Architecture.....	22
3.9 Security Architecture	22
Chapter 4: Mathematical Modeling and Recommendation Engine	24
4.1 Recommendation Problem Formulation	24
4.2 MCDM Weighted Scoring	24
4.3 Barrier Requirements.....	25
4.4 OTR	25
4.5 WVTR.....	25
4.6 Fick's Law	26
4.7 Permeability	26
4.8 Respiration Kinetics.....	26
4.9 Q10 Model	26
4.10 Lipid Oxidation.....	27
4.11 Constraint Filtering.....	27
4.12 Recommendation Ranking	28
4.13 Recommendation Pseudocode.....	28

4.14 Model Governance.....	28
Chapter 5: Implementation Details	29
5.1 Frontend Implementation	29
5.2 Main Frontend Screens	29
5.3 Backend Implementation	33
5.4 Database Access	34
5.5 Authentication and Authorization.....	34
5.6 Qdrant Integration.....	35
5.7 Groq Integration.....	35
5.8 Admin Knowledge Ingestion.....	35
5.9 Error Handling.....	36
5.10 Deployment Considerations	36
Chapter 6: Experimental Results, Testing and Validation	37
6.1 Testing Strategy.....	37
6.2 Authentication Testing.....	38
6.3 Recommendation Testing.....	38
6.4 RAG Testing	38
6.5 Database Testing.....	39
6.6 API Testing	39
6.7 Case Study 1 — Strawberries.....	39
6.8 Case Study 2 — Potato Chips	39
6.9 Case Study 3 — Fresh Chicken.....	40
6.10 Case Study 4 — Roasted Coffee Beans	40
6.11 Case Study Comparison	40
6.12 Results Reporting Template	41
Chapter 7: Conclusion, Sustainability and Future Scope	41
7.1 Conclusion.....	41
7.2 Sustainability.....	42
7.3 Limitations	42
7.4 Future Enhancements.....	43
7.5 Future Experimental Validation	43
References.....	44
Appendix A — API Examples.....	44
Appendix B — RAG Prompt Template.....	45
Appendix C — Database Schema	45
Appendix D — Testing Checklist.....	46
Appendix E — Sample Recommendation Inputs.....	46
Appendix F — Project Directory Structure	46

LIST OF TABLES

Table 1.1	Project Objectives	11
Table 2.1	Packaging Barrier Concepts	14
Table 2.2	Candidate Packaging Materials	15
Table 2.3	Literature Review Comparison	17
Table 3.1	Technology Stack	19
Table 3.2	Database Entities	21
Table 3.3	Data Dictionary Summary	22
Table 4.1	MCDM Criteria	24
Table 4.2	Example Weighting Framework	25
Table 4.3	Constraint Rules	27
Table 5.1	API Endpoints	33
Table 5.2	Database Tables	34
Table 6.1	Test Cases	37
Table 6.2	Case Study Inputs	40
Table 6.3	Case Study Evaluation Template	41
Table 7.1	Limitations and Future Improvements	43

LIST OF ABBREVIATIONS

Abbreviation	Meaning
AI	Artificial Intelligence
API	Application Programming Interface
ASTM	ASTM International
BOPP	Biaxially Oriented Polypropylene
CPP	Cast Polypropylene
CO ₂	Carbon Dioxide
EVOH	Ethylene Vinyl Alcohol
JWT	JSON Web Token
LLM	Large Language Model
MAP	Modified Atmosphere Packaging
MCDM	Multi-Criteria Decision Making
ML	Machine Learning
ONNX	Open Neural Network Exchange
OTR	Oxygen Transmission Rate
PA	Polyamide
PBAT	Polybutylene Adipate Terephthalate
PHA	Polyhydroxyalkanoate
PLA	Polylactic Acid
Qdrant	Vector Search Engine / Vector Database
RAG	Retrieval-Augmented Generation
RBAC	Role-Based Access Control
USDA	United States Department of Agriculture
VSP	Vacuum Skin Packaging
WVTR	Water Vapour Transmission Rate

CHAPTER 1: INTRODUCTION

1.1 Background

Food packaging is best understood as an engineered interface between a food product and its surroundings. The package must slow or control the transfer of moisture and oxygen, retain desirable aromas, exclude light where the product is light-sensitive, resist mechanical damage during handling and distribution, and prevent post-process contamination. For fresh produce it must also permit an appropriate exchange of respiratory gases, because the product continues to consume oxygen and release carbon dioxide after harvest. Temperature acts on every one of these processes: permeation, respiration, chemical oxidation and microbial growth all accelerate as temperature rises.

No single packaging material is best for all foods. A film that protects fried snacks from oxygen and moisture may be unsuitable for respiring berries, which need some gas exchange to avoid anaerobic conditions. A high-barrier laminate that suits vacuum-packed fresh meat may be unnecessarily costly and difficult to recycle when applied to a dry staple. Selecting a material is therefore a multi-variable engineering decision, not a lookup in a single table.

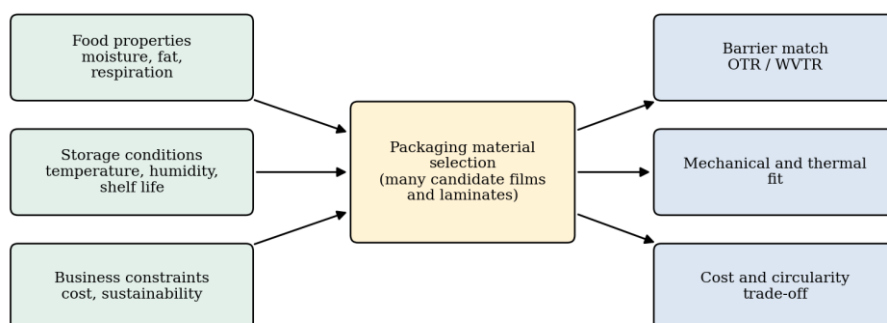


Figure 1.1: Food packaging decision challenge: several food, storage and business variables must be balanced to select a packaging material.

1.2 Problem Statement

Packaging engineers, food technologists and small food businesses select materials by combining knowledge from textbooks, supplier data sheets, standards and experience. The task is difficult to perform manually for several reasons. Food commodities span many categories, each with different moisture content, water activity, fat content and respiration behaviour. Barrier requirements differ widely: a crisp snack is limited by moisture pickup and lipid oxidation, whereas a fresh fruit is limited by gas balance and condensation. Storage temperature

changes both the product and the film. Cost and sustainability introduce further trade-offs that are rarely considered together with barrier performance.

The problem addressed by this project is the absence of an accessible, transparent decision-support tool that converts food properties and storage conditions into quantitative barrier requirements, ranks candidate packaging materials against multiple criteria, and explains the result using retrievable technical evidence rather than unsupported assertions.

1.3 Objectives

The overall aim is to design and implement PackSci AI, a decision-support platform that recommends packaging materials for food commodities. The specific technical objectives are listed in Table 1.1.

Table 1.1: Project Objectives

No.	Objective	Outcome type
O1	Build a structured database of food commodities and packaging materials with their properties.	Prototype system
O2	Derive barrier requirements (maximum allowable OTR and WVTR) from food properties and shelf-life targets.	Mathematical model
O3	Implement a deterministic Multi-Criteria Decision Making (MCDM) engine using six criteria.	Prototype system
O4	Apply hard constraints so that unsuitable materials are excluded with a stated reason.	Prototype system
O5	Provide a Retrieval-Augmented Generation (RAG) assistant grounded in a curated knowledge base.	Prototype system
O6	Store document embeddings in Qdrant and support semantic retrieval.	Prototype system
O7	Keep the deterministic engine independent of the generative language model.	Design principle
O8	Provide JWT authentication, role-based access control and private recommendation history.	Prototype system
O9	Provide a material comparison interface for side-by-side evaluation.	Prototype system
O10	Define a testing framework and a plan for future laboratory validation.	Proposed methodology

1.4 Scope of the Project

The project covers the design and implementation of a web-based prototype: a React frontend, a FastAPI backend, a PostgreSQL database hosted on Supabase, a Qdrant Cloud vector index and a Groq-hosted language model. The prototype dataset currently contains approximately 53 or more food commodity records and 20 or more packaging material records. These counts are descriptive of the current prototype and have not been independently audited.

The project does **not** include laboratory measurement of packaging permeability, shelf-life trials, microbiological challenge testing, regulatory compliance assessment, life-cycle

assessment or certification of any package. Recommendations are decision-support outputs and must be validated by experiment before industrial use.

1.5 Motivation

Two observations motivate the work. First, the knowledge needed to choose a package is scattered across handbooks, standards and supplier documents, and it is time-consuming to consult. Second, general-purpose language models can explain packaging concepts fluently but may state barrier values or performance claims that are not supported by any source. A system that performs the numerical reasoning deterministically and uses retrieval to ground the language explanation addresses both concerns: the numbers are traceable and the narrative is tied to retrievable evidence.

1.6 Need for Intelligent Packaging Selection

Packaging affects the preservation of food, but it does not by itself determine food loss or waste; losses also arise from harvest practices, handling, temperature control, demand forecasting and consumer behaviour. Reports by the Food and Agriculture Organization and the United Nations Environment Programme describe food loss and waste as a multi-stage problem [1], [2]. Within that wider context, suitable packaging is one of several levers. Choosing too little protection can shorten shelf life, while choosing excessive protection increases material use and cost and may hinder recycling. An intelligent selection tool helps make this trade-off explicit and repeatable.

1.7 Project Contributions

- **MCDM engine:** a deterministic scoring and ranking engine over six criteria with configurable weights.
- **Structured database:** PostgreSQL tables for commodities, materials, specifications, recommendations and data sources.
- **RAG system:** a retrieval pipeline that grounds the language model in curated technical documents.
- **Qdrant retrieval:** a 384-dimensional vector index using cosine similarity for semantic search.
- **LLM explanation:** a Groq-hosted model constrained by a system prompt that forbids invented values.
- **Authentication:** bcrypt password hashing, JWT bearer tokens and role-based access control.

- **Recommendation history:** per-user storage of past recommendations.
- **Material comparison:** a side-by-side interface for selected materials.

1.8 Report Organization

Chapter 2 reviews the theory of barrier packaging, permeation, respiration and retrieval-augmented generation. Chapter 3 presents requirements and architecture. Chapter 4 develops the mathematical model and the recommendation engine. Chapter 5 describes implementation, including screenshots of the working prototype. Chapter 6 defines the testing framework and four case studies without claiming experimental results. Chapter 7 concludes with sustainability considerations, limitations and future work.

Prototype limitations

The system is a prototype. Its dataset is small, its material properties are reference or illustrative values, and no laboratory validation has been carried out. Barrier performance depends on actual film structure, thickness, temperature and humidity, so results should be treated as candidates for evaluation rather than final specifications.

Chapter summary

This chapter introduced food packaging as an engineered interface, stated the selection problem, listed the objectives and defined the scope. The next chapter reviews the technical foundations.

CHAPTER 2: LITERATURE REVIEW

2.1 Food Packaging and Shelf Life

Shelf life is the period during which a food remains safe and acceptable in quality under defined storage conditions. It is limited by the fastest of several deterioration mechanisms: microbial growth, enzymatic activity, moisture gain or loss, chemical oxidation, physical damage and loss of volatile aroma. Packaging influences each mechanism by controlling mass transfer, light exposure, headspace composition and physical protection [3]. The relationship between packaging and shelf life is therefore product-specific: the dominant deterioration mechanism determines which barrier property matters most.

2.2 Oxygen and Moisture Barrier Requirements

Oxygen and water vapour are the two permeants most often controlled by packaging. Oxygen drives lipid oxidation, pigment degradation and the growth of aerobic spoilage organisms, whereas water vapour drives texture change, caking, and the growth of moulds in dry products. The barrier required is expressed as a maximum allowable transmission rate that keeps the product within its acceptable quality limit for the target shelf life. Table 2.1 summarises the concepts used throughout this report.

Table 2.1: Packaging Barrier Concepts

Concept	Meaning	Typical role in selection
OTR	Oxygen transmission rate through a film under stated test conditions.	Limits oxidation; may need a lower bound for respiring produce.
WVTR	Water vapour transmission rate through a film under stated conditions.	Limits moisture pickup or loss.
Permeability (P)	Product of diffusivity and solubility; thickness-independent property.	Allows comparison across thicknesses.
Thickness	Film or laminate thickness in micrometres.	Transmission rate varies inversely with thickness for a homogeneous film.
Headspace	Gas volume inside the sealed package.	Sets the buffer for gas exchange in MAP.

2.3 Polymer Packaging Materials

Flexible food packaging is built from a small family of polymers and laminates. Low-density polyethylene (LDPE) and high-density polyethylene (HDPE) are widely used as heat-sealable, moisture-resistant layers with relatively high oxygen permeability. Biaxially oriented polypropylene (BOPP) and cast polypropylene (CPP) provide good moisture barrier and clarity and are frequently combined in laminates. Polyethylene terephthalate (PET) offers stiffness

and moderate gas barrier. Polyamide (PA, nylon) provides mechanical strength and better oxygen barrier than polyolefins. Ethylene vinyl alcohol (EVOH) is a high oxygen-barrier layer whose performance is sensitive to humidity and is therefore usually embedded between moisture-resistant layers. Polylactic acid (PLA) and polybutylene adipate terephthalate (PBAT) are bio-based or biodegradable options with different barrier and processing characteristics. Metallized films and aluminium-foil laminates provide the highest barrier to light, oxygen and moisture but reduce recyclability [3].

Precise barrier values depend on grade, thickness, orientation, coating, temperature and humidity, and must come from supplier data or measurement. In this report the values shown by the prototype are described as reference or prototype values, and no new specification values are claimed.

Table 2.2: Candidate Packaging Materials

Material family	General characteristics (qualitative)	Typical concerns
LDPE / HDPE	Heat-sealable, tough, good moisture resistance; high oxygen permeability.	Poor oxygen barrier.
BOPP / CPP	Clear, stiff, good moisture barrier; used in laminates.	Moderate oxygen barrier unless coated or metallized.
PET	Stiff, clear, moderate gas barrier.	Often laminated for sealing.
PA / Nylon	Strong, puncture resistant, better oxygen barrier.	Moisture sensitivity; cost.
EVOH	Very high oxygen barrier in dry conditions.	Loses barrier at high humidity; needs protective layers.
PLA / PBAT	Bio-based or compostable options.	Higher moisture permeability; end-of-life infrastructure.
Metallized films	High light, oxygen and moisture barrier at low added mass.	Limited recyclability; flex cracking.
Aluminium foil laminates	Near-complete barrier.	Cost; non-recyclable structures.

2.4 Modified Atmosphere Packaging

Modified atmosphere packaging (MAP) alters the gas composition inside the package, typically by reducing oxygen and adding carbon dioxide or nitrogen, to slow deterioration. For fresh produce, equilibrium modified atmosphere packaging relies on the balance between respiration inside the package and gas permeation through the film. The resulting atmosphere depends on film permeability and area, headspace, product mass, respiration rate and temperature. Because these factors are specific to each package, this report does not state exact gas mixtures for any case study; they must be determined for the actual product and package.

2.5 Food Respiration

Fresh fruits and vegetables remain metabolically active after harvest. Respiration consumes oxygen and sugars and releases carbon dioxide, water and heat. Rates differ substantially among commodities and rise with temperature, so cold storage is a primary means of extending postharvest life [4]. If a package is too impermeable, oxygen may fall to levels that trigger fermentation and off-odours; if it is too permeable, the protective effect of a modified atmosphere is lost. Packaging for respiring produce therefore requires a window of acceptable permeability rather than a maximum barrier.

2.6 Food Oxidation

Lipid oxidation is a chain reaction in which unsaturated fatty acids react with oxygen to form hydroperoxides and secondary products that cause rancid flavours [5]. High-fat, low-moisture foods such as fried snacks and roasted nuts are particularly sensitive. Oxidation rate depends on oxygen availability, temperature, light, pro-oxidant metals and antioxidants. A package with a low oxygen transmission rate, and sometimes an opaque or metallized layer, slows the process.

2.7 AI in Food Packaging

Artificial intelligence can support packaging work in several ways: classifying food condition from images, predicting shelf life from sensor data, optimising material selection, and retrieving technical knowledge. Each use requires suitable data. Predictive shelf-life models need measured storage trials, which are not available in this project; therefore PackSci AI does not claim a trained predictive model. Instead it uses transparent engineering equations and multi-criteria ranking [6], [7], with AI applied to knowledge retrieval and explanation.

2.8 Retrieval-Augmented Generation

Retrieval-augmented generation combines a language model with an external retrieval component. Given a query, the system retrieves relevant passages from a document store and supplies them to the model as context, so that answers can be grounded in and cited to source text [8]. Retrieval commonly uses dense vector embeddings and a similarity measure such as cosine similarity. RAG reduces, but does not eliminate, the risk of unsupported statements, and its quality depends on the corpus, chunking strategy, retrieval settings and prompt design. In PackSci AI the embedding model is BAAI/bge-small-en-v1.5, which produces 384-dimensional vectors [9].

2.9 Research Gap

Reference works describe barrier theory, and commercial tools exist for specific tasks, but there is limited accessible support that combines food-specific requirement derivation, transparent multi-criteria ranking including sustainability and cost, and evidence-grounded explanation in one system. Table 2.3 compares the categories of sources used in this report with the gap that PackSci AI addresses.

Table 2.3: Literature Review Comparison

Source category	Focus	Relevance to PackSci AI	Limitation for this project
Packaging textbooks [3]	Principles of barrier, materials and MAP	Theory and material families	No decision automation
Diffusion theory [10]	Mathematics of permeation	Fick-law formulation	Requires measured parameters
Postharvest references [4]	Respiration and storage of produce	Respiration and Q_{10} reasoning	Values vary by cultivar and maturity
MCDM literature [6], [7]	Ranking with multiple criteria	Weighted scoring method	Weights are subjective
RAG research [8]	Grounded language generation	Evidence-grounded explanation	Quality depends on corpus
Test standards [11], [12]	Measurement of OTR and WVTR	Future laboratory validation	Do not validate the software

CHAPTER 3: SYSTEM REQUIREMENTS AND ARCHITECTURE

3.1 Functional Requirements

The functional requirements were derived from the objectives in Chapter 1. The system shall:

1. allow users to register, log in and log out securely;
2. let a user select a food commodity or enter custom food properties;
3. accept storage temperature, storage condition and shelf-life target;
4. derive barrier requirements from the inputs using documented equations;
5. filter and rank packaging materials using the MCDM engine and return a ranked list with scores and exclusion reasons;
6. save each recommendation to the user's private history;
7. let users compare selected materials side by side;
8. answer packaging questions through a RAG assistant that cites retrieved sources;
9. allow administrators to upload knowledge documents for ingestion; and
10. return a deterministic recommendation even if the retrieval or language-model service is unavailable.

3.2 Non-Functional Requirements

- **Security:** passwords hashed with bcrypt, JWT authentication, role-based authorization and backend-only secrets.
- **Separation of concerns:** deterministic ranking must not depend on the language model.
- **Transparency:** every recommendation must expose its inputs, criterion scores and exclusion reasons.
- **Maintainability:** modular routers, schemas and services; configurable weights and constraints.
- **Availability:** graceful degradation when Qdrant or Groq is unreachable.
- **Usability:** responsive interface with clear navigation and suggested questions.
- **Performance:** interactive response times for recommendation requests; RAG latency depends on the external LLM service and is not claimed here.

3.3 Technology Stack

Table 3.1: Technology Stack

Layer	Technology	Purpose
Frontend	React 18, TypeScript, Vite, Tailwind CSS, Recharts, Lucide Icons	User interface, forms, charts and icons
Backend	Python, FastAPI, Pydantic v2, SQLAlchemy 2.0	REST API, validation, ORM
Relational database	PostgreSQL on Supabase	Structured records and history
Vector database	Qdrant Cloud	Similarity search over knowledge chunks
Embeddings	FastEmbed / ONNX, BAAI/bge-small-en-v1.5 (384-d)	Text-to-vector conversion
LLM	Groq Cloud API (model set by configuration)	Evidence-grounded explanation
Security	bcrypt, JWT, RBAC	Authentication and authorization

The language model is a configuration setting. Candidate models available through Groq, such as Llama 3.3 70B, may be selected; the model actually deployed for a given run should be recorded in the environment configuration and confirmed before reporting results. Further details of each component are available in its official documentation [13]–[19].

3.4 Overall System Architecture

A request begins with the user in the React frontend. The frontend calls the FastAPI backend, which authenticates and validates the request. The recommendation engine reads structured data from PostgreSQL and computes requirements, constraints and scores. For explanatory questions, the backend embeds the query, searches Qdrant, cross-references PostgreSQL and sends grounded context to the Groq model. The response returns to the frontend as an evidence-grounded explanation. The deterministic recommendation logic does not depend on the language model: if retrieval or generation fails, the ranked list is still returned.

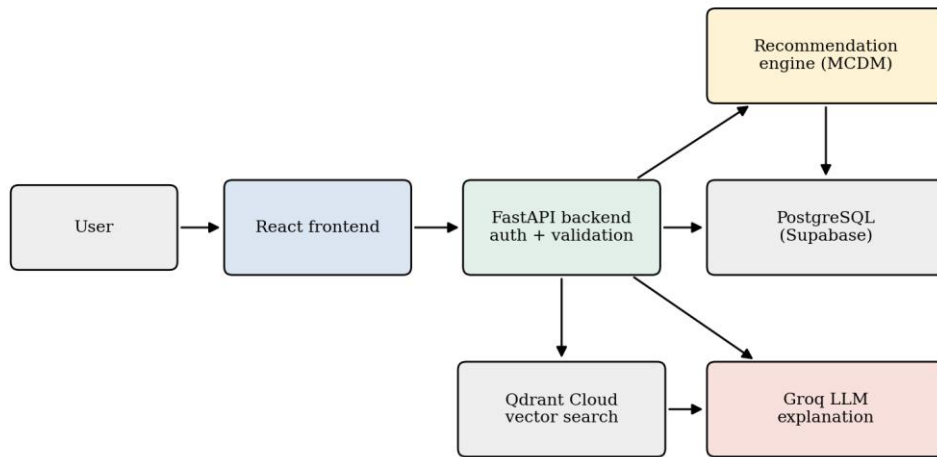


Figure 3.1: Overall PackSci AI architecture.

The system is organised in layers, shown in Figure 3.2, so that each concern can be tested and modified independently.

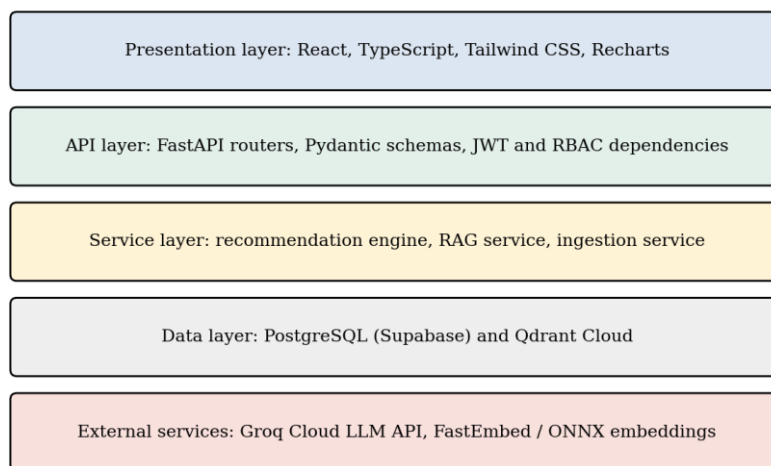


Figure 3.2: Layered system architecture.

3.5 Frontend Architecture

The frontend is a single-page application written in TypeScript with React 18 and built with Vite. Tailwind CSS provides utility-based styling, Recharts renders comparison charts and Lucide supplies icons. The source is organised into components, pages, services and types. A service layer wraps API calls and attaches the bearer token; typed interfaces mirror the backend schemas so mismatches are caught at compile time. Route guards keep authenticated pages such as recommendation history behind login.

3.6 Backend Architecture

The backend uses FastAPI with Pydantic v2 models for request and response validation and SQLAlchemy 2.0 for database access. Routers group endpoints by domain (authentication, commodities, materials, recommendation, RAG and administration). Dependency injection supplies the database session and the current user; role checks are implemented as dependencies so that protected routes fail early with 401 or 403 responses. Business logic lives in service modules, keeping routers thin.

3.7 Database Architecture

PostgreSQL, hosted on Supabase, stores structured data. Table 3.2 lists the entities and Figure 3.3 shows their relationships.

Table 3.2: Database Entities

Entity	Description	Key relationships
users	Registered accounts with role	One user has many recommendations
food_commodities	Food records and properties	One commodity has many recommendations
packaging_materials	Candidate packaging materials	One material has one or more specifications
material_specifications	Barrier and physical properties	Belongs to a material
recommendations	A saved recommendation request	Belongs to a user and a commodity
recommendation_materials	Ranked materials for a recommendation	Links recommendations and materials
data_sources	Provenance of records	Referenced by commodity and material records

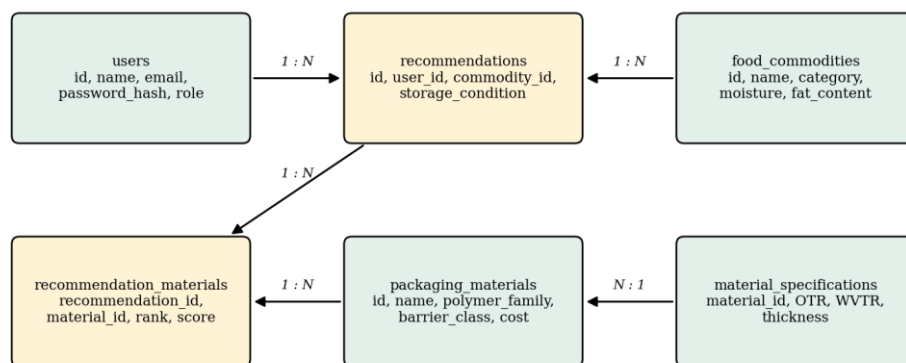


Figure 3.3: Relational data model (simplified).

Table 3.3: Data Dictionary Summary

Table	Principal columns	Notes
users	id, name, email, password_hash, role, created_at	Email unique; role used by RBAC
food_commodities	id, name, category, moisture, water_activity, respiration_rate, temperature, fat_content, oxygen_sensitivity	Values are reference or prototype data
packaging_materials	id, name, polymer_family, barrier_class, cost, sustainability_score	Cost and score are prototype estimates
material_specifications	material_id, OTR, WVTR, thickness, recyclability	Units and test conditions stored with values
recommendations	id, user_id, commodity_id, storage_condition, created_at	Filtered by user id on read
data_sources	id, name, type, reference	Supports provenance labelling

The prototype dataset currently contains approximately 53 or more food commodity records and 20 or more packaging material records. USDA FoodData Central is a possible public source of compositional data, but this report does not claim that every record has been validated against it; each record should carry a provenance label in the data_sources table.

3.8 Vector Database Architecture

Qdrant Cloud stores embeddings of knowledge chunks. Each point holds a 384-dimensional vector and a payload containing the source document, chunk index and text. The collection uses cosine distance. Point identifiers are deterministic UUIDs derived from the document identifier and chunk index, so re-ingesting the same document overwrites rather than duplicates points.

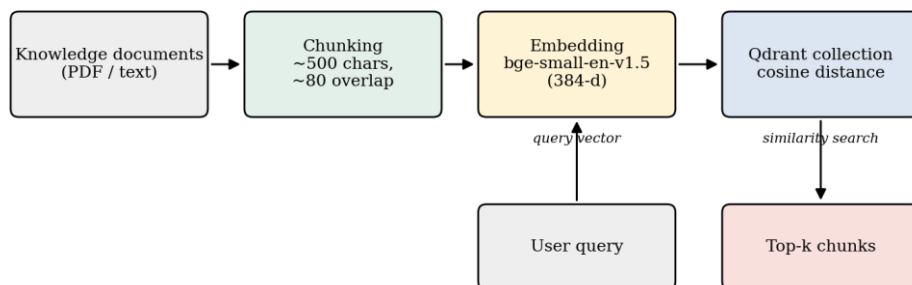


Figure 3.4: Vector embedding and retrieval pipeline.

3.9 Security Architecture

Security is layered. Passwords are hashed with bcrypt and never stored in plain text. Successful login returns a signed JWT that the client sends as a bearer token. Role-based access control restricts administrative endpoints such as knowledge upload. Recommendation history queries are always filtered by the authenticated user, preventing access to other users' records. Pydantic validates all inputs. The Groq API key, database credentials and Qdrant key are stored only in backend environment variables; the frontend never receives them. Cross-origin requests are limited by a CORS allow-list, and errors are returned without stack traces or secrets.

CHAPTER 4: MATHEMATICAL MODELING AND RECOMMENDATION ENGINE

4.1 Recommendation Problem Formulation

Let $M = \{m_1, \dots, m_n\}$ be the set of candidate materials and let x denote the input vector describing a food and its storage condition. The engine first derives requirement values from x , then removes materials that violate hard constraints, then scores the remaining materials on six criteria and ranks them. The output is an ordered list with scores and reasons. This procedure is deterministic: identical inputs and configuration produce identical outputs.

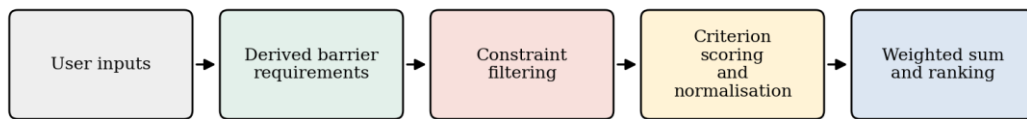


Figure 4.1: Multi-criteria recommendation pipeline.

4.2 MCDM Weighted Scoring

Six criteria are used, listed in Table 4.1. The total score of material j is the weighted sum

$$S_{total,j} = \sum_{i=1}^6 w_i \times S_{ij} \quad (4.1)$$

with $\sum w_i = 1$ and $0 \leq S_{ij} \leq 1$. The weighted-sum model is a standard multi-attribute method [6]; weight elicitation methods such as pairwise comparison [7] may be used to set weights.

Table 4.1: MCDM Criteria

Criterion	Type	Basis in prototype data
C1 Moisture barrier	Benefit or window	WVTR compared with $WVTR_{max}$
C2 Oxygen barrier	Benefit or window	OTR compared with OTR_{max} (and OTR_{min} for respiring foods)
C3 Temperature / thermal compatibility	Suitability	Material temperature range versus storage temperature
C4 Mechanical performance	Benefit	Strength or puncture class
C5 Sustainability / circularity	Benefit	Sustainability score and recyclability rating
C6 Cost efficiency	Cost	Estimated cost per kg

Normalization. Criterion values are mapped to [0, 1]. For a benefit criterion with raw value x_{ij} , min-max normalization is

$$S_{ij} = (x_{ij} - \min_j x_{ij}) / (\max_j x_{ij} - \min_j x_{ij}) \quad (4.2)$$

and for a cost criterion the complement $1 - S_{ij}$ is used. Because barrier properties span orders of magnitude (for example OTR), a logarithmic transform may be applied before normalization; this is a configurable design choice.

Table 4.2: Example Weighting Framework

Food class	C1	C2	C3	C4	C5	C6	Sum
Respiring fresh produce	0.15	0.20	0.15	0.20	0.15	0.15	1.00
Fried snacks	0.25	0.30	0.05	0.10	0.10	0.20	1.00
Fresh meat (chilled)	0.10	0.30	0.20	0.20	0.05	0.15	1.00
Dry staples	0.30	0.10	0.05	0.15	0.15	0.25	1.00
Roasted coffee	0.20	0.30	0.05	0.10	0.10	0.25	1.00

The weights in Table 4.2 are an illustrative, configurable framework. They are engineering judgements, not values derived from experiments, and they should be reviewed by domain experts before use. Sensitivity analysis, in which each weight is varied and the change in ranking observed, is recommended to assess how strongly the outcome depends on them.

4.3 Barrier Requirements

The barrier requirement is the largest transmission rate that still lets the product reach its shelf-life target. It is derived from the food's tolerance to oxygen uptake and moisture change, the package area and the storage time. Sections 4.4 and 4.5 give the formulations. The tolerances themselves (critical moisture, allowable oxygen uptake) are product-specific inputs that must come from literature or testing.

4.4 OTR

The oxygen transmission rate of a film is defined as the volume of oxygen passing through unit area per unit time at a specified partial-pressure difference:

$$OTR = Q_{O_2} / (A \times t \times \Delta p_{O_2}) \quad (4.3)$$

where Q_{O_2} is the oxygen volume (cm^3), A the area (m^2), t time (days) and Δp_{O_2} the oxygen partial-pressure difference (atm). If $O_{\max}$ is the oxygen uptake the product can tolerate over the shelf life t_s , the maximum allowable OTR for a package of area A is

$$OTR_{\max} = O_{\max} / (A \times t_s \times \Delta p_{O_2}) \quad (4.4)$$

4.5 WVTR

The water vapour transmission rate is the mass of water vapour passing through unit area per unit time under stated temperature and humidity conditions:

$$WVTR = \Delta m / (A \times t) \quad (4.5)$$

For a dry product with solids mass W_s that may gain moisture from dry-basis content X_0 to a critical content X_c , the allowed water gain is $\Delta m_{\max} = W_s(X_c - X_0)$, giving

$$WVTR_{\max} = W_s(X_c - X_0) / (A \times t_s) \quad (4.6)$$

The critical moisture X_c is a product-specific quality limit, for example the onset of loss of crispness, and must be sourced or measured.

4.6 Fick's Law

Steady diffusion of a permeant through a film is described by Fick's first law [10]:

$$J = -D (dC/dx) \quad (4.7)$$

where J is the flux, D the diffusion coefficient, C the concentration of the permeant in the polymer and x the position across the thickness. For a homogeneous film of thickness L with concentrations C_1 and C_2 on its two faces, the flux is $J = D(C_1 - C_2)/L$.

4.7 Permeability

If the concentration in the polymer is proportional to the partial pressure p of the permeant (Henry's law), $C = S p$, with S the solubility coefficient, the flux becomes

$$J = P (p_1 - p_2) / L, \quad P = D \times S \quad (4.8)$$

Permeability P therefore combines a diffusion contribution D and a solubility contribution S . Because the transmission rate of a homogeneous film varies inversely with thickness, $OTR \approx P \Delta p / L$. Laminates, coatings and humidity-dependent polymers such as EVOH deviate from this simple relation, so measured values are preferred for actual structures.

4.8 Respiration Kinetics

For a sealed package of respiring produce with mass W , the oxygen balance at quasi-steady state equates oxygen ingress through the film to oxygen consumption:

$$OTR \times A \times (p_{out} - p_{in}) = R_{O_2} \times W \quad (4.9)$$

where R_{O_2} is the oxygen consumption rate per unit mass. Solving for p_{in} indicates the equilibrium oxygen partial pressure for a candidate film. The equation shows why the

acceptable OTR for produce lies in a window: a very low OTR drives p_{in} towards zero. Respiration rates R_{O_2} are commodity- and temperature-specific inputs, and gas-dependent inhibition of respiration is neglected in this simple model.

4.9 Q10 Model

The temperature dependence of a rate is often approximated by the Q_{10} relation:

$$R(T_2) = R(T_1) \times Q_{10}^{(T_2 - T_1)/10} \quad (4.10)$$

In the prototype Q_{10} is a configurable parameter, and values of 2.0 to 2.5 may be used as an engineering assumption. This range is **not** a universal value for all foods; it varies with commodity, temperature range and maturity, and should be replaced with commodity-specific data where available.

4.10 Lipid Oxidation

For oxygen-sensitive high-fat foods, a simple kinetic approximation treats the extent of oxidation as growing with time at a rate that follows an Arrhenius dependence on temperature:

$$k(T) = k_0 \exp(-E_a / (R T)) \quad (4.11)$$

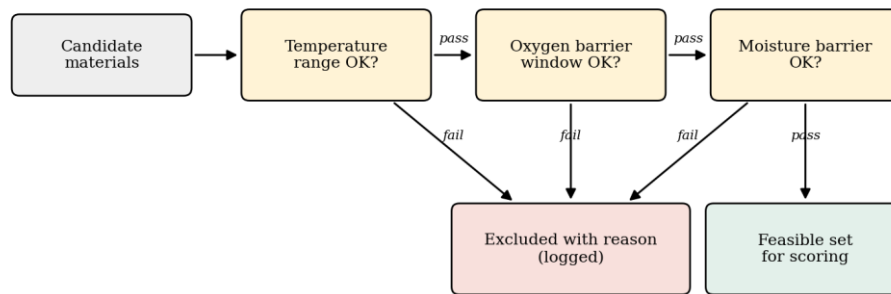
where E_a is an activation energy and R the gas constant. In PackSci AI the purpose of this relation is qualitative: to show that higher temperature and higher oxygen availability increase the requirement for oxygen barrier. Parameters for a specific product would need to be estimated from storage experiments, and the prototype does not claim predicted shelf-life values from this equation.

4.11 Constraint Filtering

Hard constraints remove materials that are unsuitable regardless of their overall score. Table 4.3 lists example rules. Excluded materials are returned with the rule that removed them, which improves transparency.

Table 4.3: Constraint Rules

Rule	Condition	Rationale
Oxygen-sensitive food	$OTR_{\text{material}} \leq OTR_{\text{max}}$	Strong oxygen barrier is required for snacks and similar products.
Respiring produce	$OTR_{\text{min}} \leq OTR_{\text{material}} \leq OTR_{\text{max}}$	Berries and similar produce should not automatically receive maximum barrier.
Moisture-sensitive dry food	$WVTR_{\text{material}} \leq WVTR_{\text{max}}$	Prevents moisture pickup.
Cold-chain product	Storage temperature within material range	Low-temperature suitability is required.
Fresh meat	Adequate mechanical class and cold-chain suitability	Mechanical integrity and temperature control are both needed.

*Figure 4.2: Constraint filtering logic.*

4.12 Recommendation Ranking

After filtering, each remaining material receives a total score from Equation 4.1. Materials are sorted in descending order; ties are broken by a documented rule, for example lower cost. The response includes the criterion scores so that a user can see why one material ranks above another. Rankings should be read as relative within the dataset, not as absolute performance predictions.

4.13 Recommendation Pseudocode

```

function recommend(food, storage, shelf_life, weights):
    req = derive_requirements(food, storage, shelf_life) # OTR_max, WVTR_max, OTR_min
    feasible, excluded = [], []
    for m in materials:
        reason = check_constraints(m, req, storage)
        if reason is None: feasible.append(m)
        else: excluded.append((m, reason))
    if feasible is empty:
        return {ranked: [], excluded: excluded, note: "no feasible material"}
    scores = normalise_criteria(feasible) # S_ij in [0,1]
    for m in feasible:
        m.total = sum(weights[i] * scores[m][i] for i in 1..6)
    ranked = sort_desc(feasible, key=total, tiebreak=cost)
    return {ranked: ranked, excluded: excluded}
  
```

4.14 Model Governance

Governance keeps the model accountable. Weights, constraint thresholds and Q_{10} values are stored as configuration with version identifiers, and each saved recommendation records the configuration used. Data records carry provenance labels (literature or reference data, prototype or illustrative value, or to be validated). Changes to weights or thresholds should be reviewed by a domain expert and re-tested with the regression cases in Chapter 6. The language model has no authority to change any numerical output.

CHAPTER 5: IMPLEMENTATION DETAILS

5.1 Frontend Implementation

The frontend provides six areas: navigation, home, recommendation form, material comparison, AI Assistant and recommendation history, together with authentication pages. A top navigation bar exposes Home, Get Recommendation, AI Assistant, Compare Materials and My Recommendations, and shows the signed-in user. The recommendation form lets the user choose a commodity preset, which pre-fills properties such as moisture and storage condition, or enter a custom food. Submitting the form calls the backend endpoint `POST /recommendation/predict`. The listing below is a representative example of how the service layer issues this request; it illustrates the approach and is not a verbatim copy of the repository.

```
// services/recommendation.ts (representative)
export interface RecommendRequest {
  commodity_id?: number;
  custom_food?: { moisture: number; fat_content: number; respiration_rate?: number };
  storage_temperature_c: number;
  storage_condition: "ambient" | "chilled" | "frozen";
  shelf_life_days: number;
}

export async function predict(req: RecommendRequest, token: string) {
  const res = await fetch(`${API_BASE}/recommendation/predict`, {
    method: "POST",
    headers: { "Content-Type": "application/json", Authorization: `Bearer ${token}` },
    body: JSON.stringify(req),
  });
  if (!res.ok) throw new Error((await res.json()).detail ?? "Request failed");
  return res.json(); // ranked materials, scores, exclusions, requirements
}
```

The response contains the derived requirements (for example OTR_{max}), a ranked list of materials with criterion scores, and a list of excluded materials with reasons. The interface presents these in a table and lets the user open the comparison page for selected materials.

5.2 Main Frontend Screens

Home and recommendation dashboard

After login the user lands on the home dashboard (Figure 5.1). It summarises the three-step workflow (enter food details, evaluate packaging requirements, compare suitable materials) and offers standard food commodity presets such as chicken, milk powder, potato chips, strawberry and wheat flour, each showing its category, moisture value and storage condition. Selecting a preset launches the recommendation form with prefilled parameters.

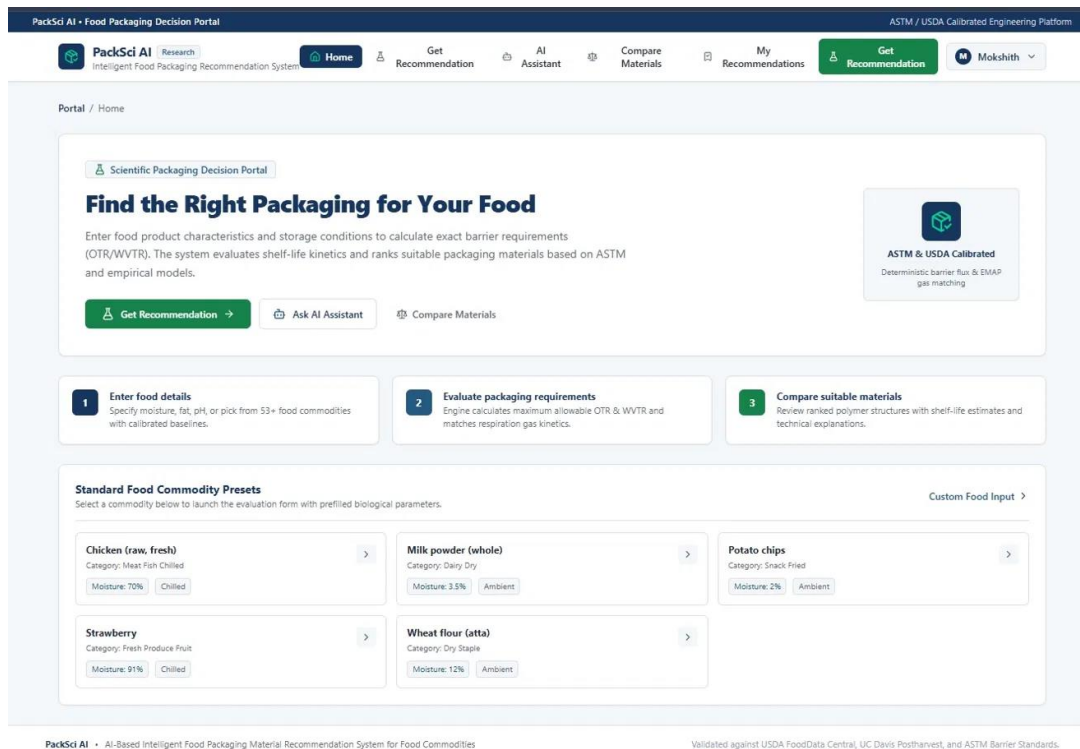


Figure 5.1: PackSci AI home and recommendation dashboard showing commodity presets (prototype data).

The banner and footer text visible in the screenshot, such as references to calibration and validation against external sources, is interface wording of the prototype. This report does not claim that the system or its dataset has been calibrated or validated against USDA, ASTM or other sources; the wording should be read as describing intended reference sources, and experimental validation remains future work (Section 7.5).

AI Assistant (RAG) interface

The AI Assistant page (Figure 5.2) provides a chat interface with a focus-context selector, suggested questions and a message history. Suggested prompts include questions about suitable packaging for wheat flour, the meaning of OTR and WVTR, and modified atmosphere packaging. The backend implementation is described in Sections 5.6 and 5.7.

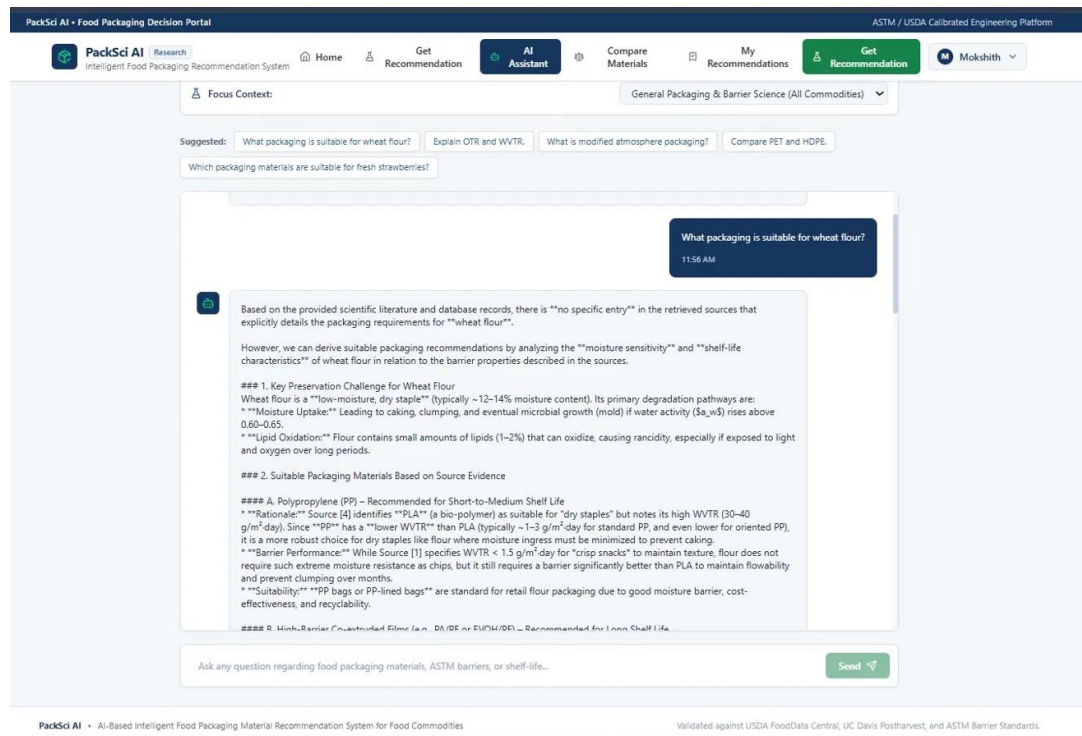


Figure 5.2: PackSci AI RAG Assistant answering a question about wheat flour (example output of the language model; not validated).

The response shown is generated text. It states that no specific entry for wheat flour was found in the retrieved sources and then reasons from related evidence, which is the intended behaviour of distinguishing evidence from inference. Numerical statements within such a response are model output and have not been verified by this project. The screenshot also shows that Markdown symbols and a mathematical expression are displayed as raw text; correct Markdown rendering is listed as a future interface improvement.

Landing page

Unauthenticated visitors see the landing page (Figure 5.3), which states the purpose of the system and offers Sign In, Create Account and Get Started actions.

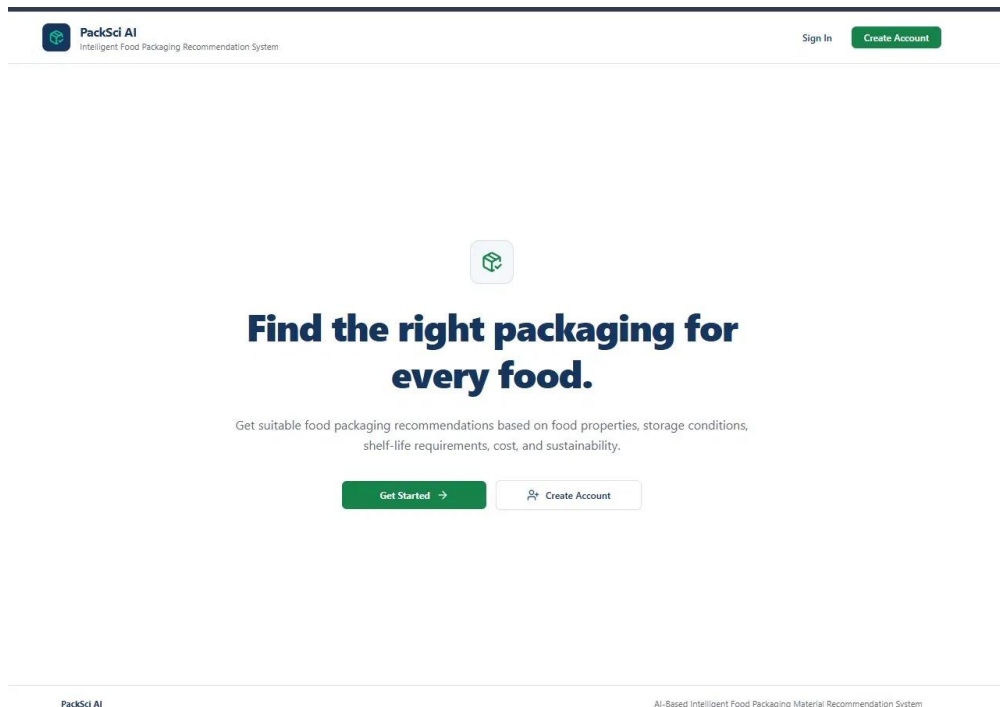


Figure 5.3: PackSci AI landing page.

Material comparison

The comparison page (Figure 5.4) lets the user toggle materials from the dataset (M01 to M20) and view a specification table containing polymer family, barrier class, OTR, WVTR, nominal thickness, estimated cost, recyclability and sustainability score. A chart plots WVTR against sustainability score for the selected materials. In the example, LDPE film, a metallised BOPP/CPP laminate and a PET/aluminium foil/PE laminate are compared, illustrating the trade-off between barrier and sustainability. The values shown are prototype system values from the project database and, as stated in Chapter 2, are not new laboratory measurements.

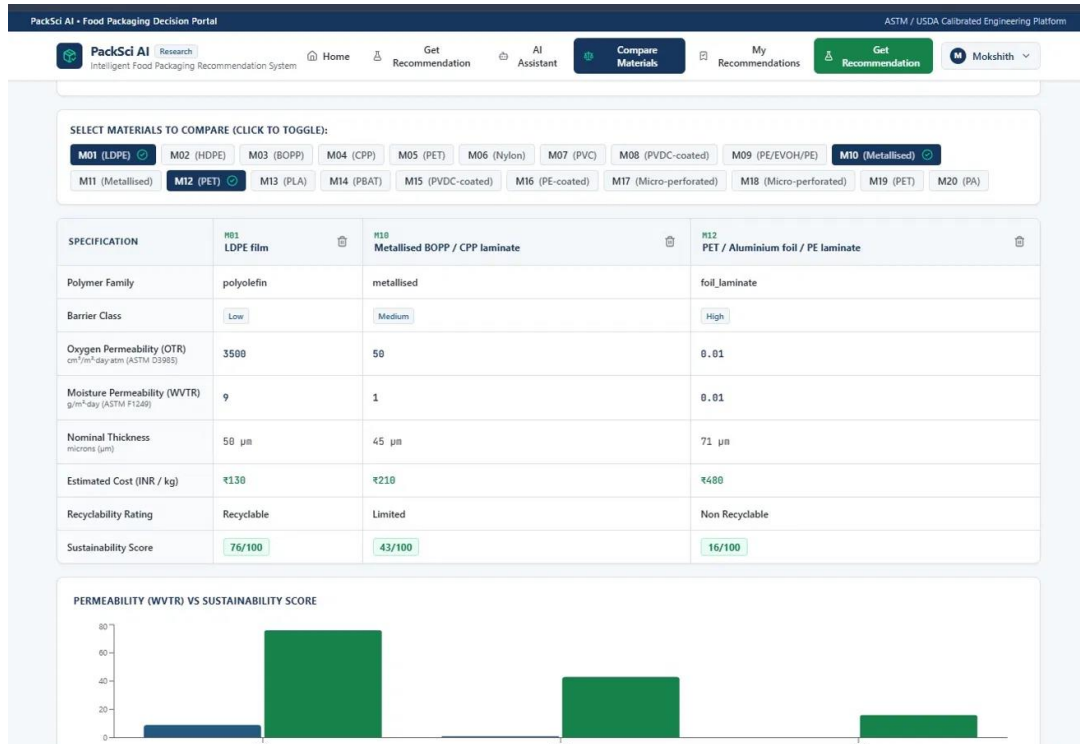


Figure 5.4: PackSci AI material comparison interface showing barrier, thickness, cost and sustainability attributes (prototype values).

5.3 Backend Implementation

The FastAPI backend exposes the endpoints listed in Table 5.1. Request bodies are validated by Pydantic models; invalid input yields a 422 response before any business logic runs. Authentication is required for all endpoints except registration, login and the public reference lists, and administrative endpoints require the admin role.

Table 5.1: API Endpoints

Method and path	Purpose	Access
POST /auth/register	Create an account	Public
POST /auth/login	Authenticate and receive a JWT	Public
GET /commodities	List food commodities	Authenticated
GET /materials	List packaging materials	Authenticated
POST /recommendation/predict	Compute ranked recommendation	Authenticated
POST /shelf-life/predict	Estimate shelf-life indicators from model equations (prototype)	Authenticated
GET /recommendations/history	Return the current user's saved recommendations	Authenticated
POST /rag/query	Answer a question using retrieval and the LLM	Authenticated
POST /admin/knowledge/upload	Ingest a knowledge document	Admin only

The shelf-life endpoint returns indicative values computed from the documented equations and configurable parameters. Its outputs are prototype estimates, not measured or validated shelf life.

5.4 Database Access

SQLAlchemy 2.0 maps tables to Python classes and executes parameterised queries, which protects against SQL injection. A session dependency opens and closes the session per request. Table 5.2 summarises the tables with their keys and indexes.

Table 5.2: Database Tables

Table	Primary key	Foreign keys	Suggested index
users	id	—	unique(email)
food_commodities	id	source_id	name, category
packaging_materials	id	source_id	polymer_family
material_specifications	id	material_id	material_id
recommendations	id	user_id, commodity_id	user_id, created_at
recommendation_materials	id	recommendation_id, material_id	recommendation_id
data_sources	id	—	name

5.5 Authentication and Authorization

Registration hashes the password with bcrypt. Login verifies the hash and issues a JWT containing the user identifier, role and expiry. A dependency decodes the token on each protected request and a second dependency checks the role. The following representative listing shows the pattern.

```
# routers/auth.py (representative)
from fastapi import APIRouter, Depends, HTTPException
from passlib.hash import bcrypt

router = APIRouter(prefix="/auth")

@router.post("/login")
def login(body: LoginIn, db: Session = Depends(get_db)):
    user = db.scalar(select(User).where(User.email == body.email))
    if not user or not bcrypt.verify(body.password, user.password_hash):
        raise HTTPException(status_code=401, detail="Invalid credentials")
    return {"access_token": make_jwt(user.id, user.role), "token_type": "bearer"}

def require_admin(user: User = Depends(current_user)):
    if user.role != "admin":
        raise HTTPException(status_code=403, detail="Forbidden")
    return user
```

5.6 Qdrant Integration

Knowledge documents are split into chunks of approximately 500 characters with an overlap of approximately 80 characters; these are implementation parameters that may be

tuned. Each chunk is embedded with BAAI/bge-small-en-v1.5 through FastEmbed and ONNX Runtime, producing a 384-dimensional vector, and stored in Qdrant with a payload. Chunk identifiers are deterministic UUIDs (for example UUID version 5 computed from the document identifier and chunk index), so re-ingestion updates existing points. At query time the question is embedded and Qdrant returns the top-k points by cosine similarity, defined for vectors u and v as

$$\cos(u, v) = (u \cdot v) / (\|u\| \|v\|) \quad (5.1)$$

```
# rag/store.py (representative)
import uuid
from qdrant_client.models import PointStruct

def chunk_id(doc_id: str, i: int) -> str:
    return str(uuid.uuid5(uuid.NAMESPACE_URL, f"{doc_id}:{i}"))

def upsert(client, coll, doc_id, chunks, vectors):
    pts = [PointStruct(id=chunk_id(doc_id, i), vector=v,
                        payload={"doc": doc_id, "i": i, "text": c})
            for i, (c, v) in enumerate(zip(chunks, vectors))]
    client.upsert(collection_name=coll, points=pts)
```

The RAG assistant shown in Figure 5.2 uses this store. RAG is used rather than relying on the language model alone because retrieved passages provide a verifiable basis for statements, allow the knowledge base to be updated without retraining, and enable citation of sources.

5.7 Groq Integration

The backend sends a system prompt and the grounded context to a Groq-hosted model over HTTPS using a server-side API key. The system prompt (Appendix B) instructs the model not to invent OTR, WVTR, material properties or laboratory results; to distinguish evidence from inference; to cite retrieved sources; to state uncertainty; and to recommend expert or laboratory validation where appropriate. If retrieval fails, the backend returns the deterministic recommendation without an LLM narrative. The frontend never receives the API key.

5.8 Admin Knowledge Ingestion

An administrator uploads a document through POST /admin/knowledge/upload. The backend checks the role, validates file type and size, extracts text, chunks it, computes embeddings, upserts the points into Qdrant and records the document in the data_sources table with a provenance label. Only documents of known origin should be ingested, because the quality of RAG answers is bounded by the quality of the corpus.

5.9 Error Handling

Errors are mapped to consistent HTTP responses: 400 or 422 for invalid input, 401 for missing or invalid credentials, 403 for insufficient role, 404 for missing resources and 503 when an external service is unavailable. External calls to Qdrant and Groq use timeouts and are wrapped so that a failure degrades the response instead of crashing the request. Error messages returned to the client contain no secrets or stack traces; details go to server logs.

5.10 Deployment Considerations

- Serve the frontend as static assets over HTTPS and the backend behind a reverse proxy.
- Keep all secrets in environment variables or a secrets manager and rotate them periodically.
- Restrict CORS to the deployed frontend origin.
- Apply database migrations in a controlled step and back up the database.
- Monitor external-service errors and latency, and set rate limits on the RAG endpoint to control cost and abuse.
- Record the model name, embedding model and configuration version used for each release.

CHAPTER 6: EXPERIMENTAL RESULTS, TESTING AND VALIDATION

6.1 Testing Strategy

This chapter defines the testing framework and the case-study inputs. **No test results or experimental measurements are reported as achieved.** Where actual results are not available, the tables state "To be executed" or "Prototype test case". A test is marked as passed only when evidence such as a log or report is recorded.

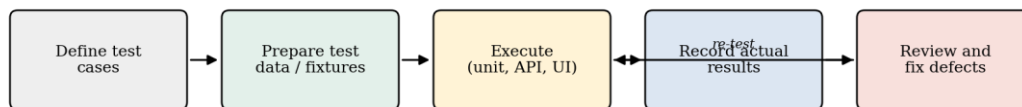


Figure 6.1: Testing and validation workflow.

Three kinds of evidence must be kept separate: literature or reference data used as inputs, prototype behaviour observed when running the software, and experimental validation of packaging performance in a laboratory. Only the first two are within the current scope.

Role of ASTM D3985 and ASTM F1249

ASTM D3985 is a test method for measuring oxygen gas transmission rate through plastic film and sheeting using a coulometric sensor, and ASTM F1249 is a test method for measuring water vapour transmission rate through plastic film and sheeting using a modulated infrared sensor [11], [12]. They are laboratory measurement methods that could be used in future to measure the OTR and WVTR of candidate packaging structures. They do **not** validate the AI model or the recommendation engine, and referencing them does not imply that any measurement has been performed in this project.

Table 6.1: Test Cases

Test ID	Test description	Input	Expected result	Actual result	Status
T01	Registration with valid data	Name, email, password	Account created; password stored hashed	To be executed	Not run
T02	Registration with duplicate email	Existing email	Rejected with error	To be executed	Not run
T03	Login with wrong password	Valid email, wrong password	401 Unauthorized	To be executed	Not run

Test ID	Test description	Input	Expected result	Actual result	Status
T04	JWT validation	Expired or tampered token	401 Unauthorized	To be executed	Not run
T05	RBAC on upload	Non-admin token	403 Forbidden	To be executed	Not run
T06	Recommendation request	Strawberry, 4 °C	Ranked list with scores and exclusions	To be executed	Not run
T07	Constraint exclusion	Oxygen-sensitive food	Materials above OTR limit excluded with reason	To be executed	Not run
T08	Recommendation persistence	Saved request	Record appears in history	To be executed	Not run
T09	History isolation	Two users	Each sees only own records	To be executed	Not run
T10	Qdrant retrieval	Query on OTR	Relevant chunks returned	To be executed	Not run
T11	RAG with retrieval failure	Qdrant unavailable	Deterministic result still returned	To be executed	Not run
T12	API input validation	Negative shelf life	422 validation error	To be executed	Not run
T13	Frontend comparison	Select three materials	Table and chart update	To be executed	Not run

6.2 Authentication Testing

Authentication tests check registration, login, token validation and role checks (T01 to T05). Additional negative tests should confirm that passwords are never returned by any endpoint, that tokens expire, and that repeated failed logins do not reveal whether an email exists.

6.3 Recommendation Testing

Recommendation tests verify determinism (identical inputs give identical outputs), constraint behaviour, normalization bounds, weight sums equal to one and tie-breaking. Regression cases based on the four case studies below should be stored so that changes to weights or thresholds are visible. Sensitivity tests vary each weight and record the ranking change.

6.4 RAG Testing

RAG testing checks that retrieved chunks are relevant, that answers cite retrieved sources, and that the model refuses to invent numerical properties when the corpus lacks them. A curated question set with reference passages should be prepared and assessed by a domain

reviewer. Retrieval accuracy and answer-quality scores are not reported here because they have not been measured.

6.5 Database Testing

Database tests confirm schema constraints, foreign-key integrity, uniqueness of emails, correct persistence of recommendations and their ranked materials, and that queries for history filter by the authenticated user. Migrations should be tested on an empty database and on a copy of existing data.

6.6 API Testing

API tests exercise every endpoint listed in Table 5.1 with valid, invalid and unauthorized requests, and check status codes and response schemas. Tools such as pytest with the FastAPI test client can automate them. Load and latency testing are separate activities and no performance figures are claimed.

6.7 Case Study 1 — Strawberries

Conditions: fresh strawberries stored chilled at approximately 4 °C. The product has high moisture content and high respiration. Respiration consumes oxygen and releases carbon dioxide and heat, so the package must permit gas exchange. High humidity inside the package can cause condensation that promotes mould, and soft fruit is vulnerable to mechanical damage.

A high-barrier film would allow oxygen to fall too low and could encourage anaerobic metabolism, which is why the prototype uses an OTR window for respiring produce. Micro-perforation is one way to tune gas exchange and reduce condensation. A literature-informed candidate is a micro-perforated BOPP or PLA-type structure. This is a prototype candidate and requires experimental validation; no gas composition is prescribed here, because the equilibrium atmosphere depends on fruit mass, film area, perforation, temperature and respiration rate.

6.8 Case Study 2 — Potato Chips

Conditions: fried potato chips at approximately 25 °C ambient storage. The product has low moisture, high fat content and oxygen sensitivity. Two mechanisms limit shelf life: lipid oxidation, which produces rancid flavours, and moisture pickup, which destroys crispness. Both are slowed by low oxygen and water vapour transmission, and light exposure can accelerate oxidation.

A literature-informed candidate is a metallized BOPP/PP-type laminate, which combines low moisture permeability with a metallized barrier layer. This is a candidate for evaluation; exact OTR and WVTR values are not asserted in this report and must come from supplier data or measurement. Package nitrogen flushing and seal integrity also influence results but are outside the software model.

6.9 Case Study 3 — Fresh Chicken

Conditions: highly perishable, high-moisture fresh chicken in the cold chain. The package must limit oxygen ingress, remain intact through handling and hold up at low temperature. A candidate structure is an EVOH/PA vacuum skin pack, in which an oxygen-barrier layer is combined with a strong polyamide layer that conforms to the product.

Microbial safety depends on hygiene, temperature control, initial contamination, and regulatory requirements. PackSci AI does not validate microbial safety and must not be used as evidence of it. Package selection for meat should be reviewed against applicable food-contact and food-safety regulations and confirmed by shelf-life and microbiological testing.

6.10 Case Study 4 — Roasted Coffee Beans

Conditions: roasted coffee is sensitive to oxygen, which degrades aroma and flavour, and it releases carbon dioxide after roasting (degassing). A sealed high-barrier package without relief could swell, while excessive gas exchange would allow aroma loss and oxidation. A candidate is a high-barrier valve package: a barrier laminate fitted with a one-way degassing valve.

Valve selection, roast level, degassing profile and package fill all affect performance, so valve type and actual shelf life require validation. The software can rank barrier laminates but cannot select a valve.

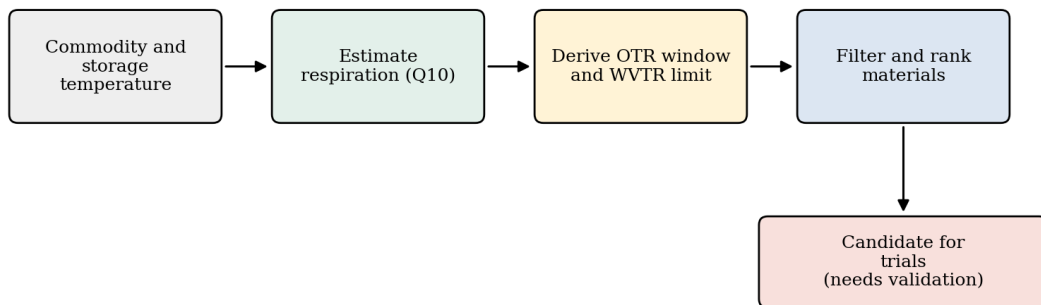
6.11 Case Study Comparison

Table 6.2 summarises the inputs and the dominant deterioration mechanism for each case. It contains qualitative descriptions only.

Table 6.2: Case Study Inputs

Case	Storage	Moisture / fat	Dominant concern	Candidate type (to be validated)
Strawberries	≈ 4 °C chilled	High moisture	Respiration, condensation	Micro-perforated BOPP / PLA-type
Potato chips	≈ 25 °C ambient	Low moisture, high fat	Oxidation, moisture pickup	Metallized BOPP/CPP-type laminate
Fresh chicken	Cold chain	High moisture	Oxygen, integrity, microbial safety	EVOH/PA vacuum skin pack
Roasted coffee	Ambient	Low moisture	Oxygen, CO ₂ degassing	High-barrier valve package

The comparison shows why different foods lead to different candidates: respiring produce needs controlled gas exchange, fried snacks need strong oxygen and moisture barrier, fresh meat needs oxygen barrier and mechanical robustness, and coffee needs barrier plus gas relief.

*Figure 6.2: Fresh produce packaging decision workflow.*

6.12 Results Reporting Template

When real evidence is collected, results should be reported with the template in Table 6.3. Each entry should state whether it comes from literature, prototype output or experiment, and link to the raw data.

Table 6.3: Case Study Evaluation Template

Item	Description	Value / evidence
Food and storage	Commodity, temperature, humidity	To be recorded
Package structure	Layers, thickness, supplier	To be recorded
Measured OTR	ASTM D3985 conditions and result	To be measured
Measured WVTR	ASTM F1249 conditions and result	To be measured
Shelf-life endpoint	Quality limit and method	To be defined
Prototype ranking	Rank and score from software	From software log
Agreement / discrepancy	Comparison of ranking with measured performance	To be analysed

CHAPTER 7: CONCLUSION, SUSTAINABILITY AND FUTURE SCOPE

7.1 Future Experimental Validation

A validation plan would select several commodities, prepare candidate packages ranked by the software, measure OTR and WVTR according to ASTM D3985 and ASTM F1249, store packages under controlled conditions and monitor agreed quality endpoints such as headspace gas, moisture, sensory quality or oxidation markers. Agreement between the predicted ranking and measured performance would then be analysed and used to revise weights and constraints. Until such work is done, all recommendations remain hypotheses for testing.

7.2 Sustainability

Packaging optimisation has the potential to support food-loss reduction, material optimisation, recyclability, circularity and resource efficiency, but these outcomes are not guaranteed by software. An AI tool does not automatically reduce food waste; benefits arise only if better packaging decisions are actually implemented and validated. The environmental impact of a package should also be weighed against the impact of the food it protects: the embedded resources in a food are often greater than those in its packaging, so a modest increase in material can be justified if it prevents loss.

There is a two-sided trade-off. Excessive packaging wastes material and may complicate recycling, whereas insufficient packaging can allow spoilage and waste both the food and the package. The prototype expresses part of this trade-off through its sustainability and recyclability criteria, as in the comparison of LDPE film with high-barrier foil laminates. A rigorous assessment requires life-cycle assessment, which is future work.

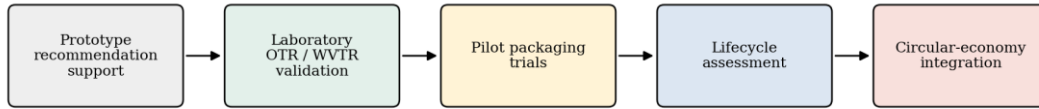


Figure 7.1: Sustainability and circular-economy roadmap.

7.3 Limitations

The main limitations are summarised in Table 7.1 with corresponding improvements.

Table 7.1: Limitations and Future Improvements

Limitation	Effect	Future improvement
Small prototype dataset	Limited coverage of foods and structures	Expand and audit dataset with provenance
No laboratory validation	Rankings unverified against measured performance	ASTM D3985 / F1249 testing and pack trials
Barrier depends on actual film structure	Reference values may not match a supplier grade	Use supplier or measured data per structure
Temperature and humidity dependence	Barrier values change with conditions	Store condition-specific values
Food variability and package geometry	Respiration and MAP results vary	Commodity-specific and geometry-aware models
Cost estimates change	Cost score becomes outdated	Regularly updated price inputs
Sustainability score is simplified	Not a life-cycle result	Life-cycle assessment integration
LLM and retrieval limits	Possible incomplete or unsupported statements	Evaluation set, citation checks, Markdown rendering

7.4 Future Enhancements

- Laboratory OTR and WVTR testing of candidate structures.
- Real packaging and shelf-life trials.
- More food commodities and packaging structures, with audited provenance.
- Improved machine-learning models once measured data exist.
- Computer vision for assessing food condition.
- IoT sensors and real-time temperature monitoring.
- A digital twin of the cold chain and package.
- Life-cycle assessment and circularity metrics.

- Industrial pilot deployment with domain-expert review.

7.5 Conclusion

This report presented PackSci AI, a decision-support prototype that integrates packaging engineering with artificial intelligence. A multi-criteria decision-making engine provides deterministic ranking of packaging materials from derived barrier requirements and hard constraints. A retrieval-augmented generation pipeline provides evidence-grounded explanations, with Qdrant enabling semantic retrieval over a curated knowledge base. PostgreSQL stores structured records, FastAPI provides secured backend services, and a React interface presents recommendations, comparisons and an AI assistant.

The key design decision is the separation of the deterministic engine from the generative layer: numerical results are traceable and reproducible, while the language model only explains and cites. The report deliberately distinguishes literature or reference data, prototype behaviour and experimental validation, and it makes no claim of laboratory validation, predictive accuracy or measured shelf-life improvement.